

06 Concurrency & Network 开卷速查

06 Concurrency & Network 开卷速查

先把整章捡回来

这一章可以先用一句话串起来：

一个 client 通过 socket 连到 server；如果 client 在私网里，SNAT router 会改 packet 的 source address；server 为了同时服务多个 client，会用 prethreaded worker pool；master 和 workers 之间用 bounded buffer 传 connfd，这个 buffer 必须用 semaphore / lock 同步。

如果完全忘了网络，先把它想成“两个程序互相发字节流”。

client 想和 server 说话，不能只知道“那台机器在哪里”，还要知道“那台机器上哪个程序在听”。所以地址要写成 IP:port：

- IP 找到 host / machine。
- port 找到 host 上的某个 service / process。
- IP:port 合起来叫 socket address。

一次 TCP connection 有两端：

client endpoint: client_ip:client_port
server endpoint: server_ip:server_port

这两个 endpoint 合起来就是 socket pair：

(client_ip:client_port, server_ip:server_port)

对程序来说，socket 看起来像 file descriptor。连接建好后，client 和 server 都可以对 socket descriptor 做 read/write，就像读写文件一样。

建立连接时，两边分工不同：

- client 主动调用 connect，说“我要连这个 server address”。

- server 先准备一个 `listenfd`，绑定自己的 IP:port，进入 listen 状态。
- server 调 `accept(listenfd, ...)` 阻塞等连接。
- 有 client 连上后，`accept` 返回一个新的 `connfd`。
- `listenfd` 继续负责等下一个 client；`connfd` 负责和当前这个 client read/write。

SNAT 解决的是“私网地址不能直接在公网路由”的问题。比如 192.168.1.10 是私有地址，很多宿舍/公司/学校局域网里都可能有同一个 192.168.1.10，公网 server 无法把它当作全球唯一地址。于是 router 出门时把 packet 的 source 从 private IP:port 改成 router 的 public IP:port。server 看到的是 router public address，不是内网 client 的 private address。回包时 router 再根据 NAT table 改回 private address。

Concurrency 部分解决的是“server 怎么同时服务多个 client”。最简单的 server 一次只服务一个 client：如果这个 client 很慢，后面的 client 全都等着。并发 server 用多个 thread 同时处理多个 `connfd`。但如果每来一个连接就创建一个 thread，开销和数量都不好控制。所以 prethreaded server 启动时先创建固定数量 worker threads，形成 thread pool。

prethreaded server 的结构是：

master thread:

accept connection -> 得到 connfd -> 放进 bounded buffer

worker threads:

从 bounded buffer 取 connfd -> echo(connfd) -> close(connfd)

这里 bounded buffer 是共享数据结构，master 和多个 workers 会同时访问，所以必须同步：

- slots 记录还有多少空位。
- items 记录 buffer 里有多少 connfd。
- mutex 保护 buffer 的 front/rear/buf 不被多个 thread 同时改坏。

lock / semaphore 的核心不是“让程序更快”，而是“让并发 interleaving 不会把共享状态弄坏”。但同步正确不等于公平：FIFO buffer 可以保证已经入队的 `connfd` 按 FIFO 出队，却不保证 semaphore 按 FIFO 唤醒等待的 worker。

本章考试大概率不是让你自由发挥系统设计，而是给一个类似

EXE/concurrent&network.pdf 的综合题，按 Part 填表、解释、写同步伪代码。

关键名词解释

名词	先这样理解
host	一台机器，比如你的电脑、server、router
client	主动发起连接的一方
server	等别人连接并提供服务的一方
IP address	找到一台 host 的网络地址
port	host 上某个 service / process 的编号
socket address	IP:port，比如 202.120.40.85:15213
endpoint	一条连接的一端，也是一个 socket address
socket pair	一条 TCP 连接的两端： (client_ip:client_port, server_ip:server_port)
socket descriptor	程序里代表 socket 的整数描述符，可 read/write
listenfd	server 用来等待新连接的 listening descriptor
connfd	accept 返回的 connected descriptor，用来服务某个 client
router	转发 packet 的网络设备
private IP	私网地址，如 192.168.x.x，公网不能直接路由
public IP	公网地址，可以在公网路由
NAT table	router 维护的 private address 和 public mapping 的表
SNAT	Source NAT，出公网时改 source IP:port
thread	同一进程里的并发执行流
critical section	访问共享状态、必须互斥执行的代码
semaphore	同步变量；可做资源计数，也可做互斥
mutex	互斥锁，只允许一个 thread 进入 critical section

名词	先这样理解
bounded buffer	固定大小队列，用来在 producer 和 consumer 间传东西
producer-consumer	一个放东西，一个取东西；本题 master 放 connfd，workers 取 connfd
prethreaded server	启动时预先创建 worker pool 的并发 server

一分钟速查

CON&NET 题的核心不是 signal，也不是 system-level I/O。复习课明确说 signal 期末完全不考，system-level I/O 大概率不考。真正要准备的是：socket API、SNAT、bounded buffer / semaphore、prethreaded server、lock 基本语义。

来源优先级：

- EXE/concurrent&network.pdf：最核心样题，SNAT and Prethreaded Server。
- EXE/concurrent&network_sol.pdf：对应答案。
- courseware/2-20-network_*.pdf：network / socket / SNAT 基础。
- courseware/2-21-con_*.pdf、2-22-sync_*.pdf：thread / semaphore / bounded buffer。
- courseware/2-23-locks_*.pdf：lock、fairness、starvation。
- courseware/2-24-cv_*.pdf：condition variable，只作背景。

题型对应来源：

题型	主要来源
connect / bind / listen / accept、listenfd VS connfd	EXE/concurrent&network.pdf Part 1
SNAT 地址转换表、server 为什么看不到 private IP	EXE/concurrent&network.pdf Part 2
bounded buffer、semaphore 初值、P/V 顺序、deadlock 解释	EXE/concurrent&network.pdf Part 3
prethreaded server 相比 iterative / thread-per-connection 的优势	EXE/concurrent&network.pdf Part 4
worker 数量 N、buffer 大小 B、CPU-bound vs I/O-bound	EXE/concurrent&network.pdf Part 5

题型	主要来源
FIFO buffer、公平性、starvation、semaphore wakeup 是否 FIFO	EXE/concurrent&network.pdf Part 6
TAS / ticket lock / sleep while holding lock	courseware/2-23-locks_*.pdf

核心句：

这题基本就是“网络连接怎么建立 + NAT 地址怎么改 + 多线程服务器如何安全地把 connfd 分发给 worker”。

Part 1: Socket API

来源：EXE/concurrent&network.pdf Part 1。

connect

客户端调用：

```
connect(clientfd, server_addr, server_addrlen)
```

作用：请求内核和 server socket address 建立 TCP connection。成功后，clientfd 成为客户端连接端点，可以用普通 Unix I/O 读写。

bind

服务器调用：

```
bind(listenfd, server_addr, server_addrlen)
```

作用：把 socket descriptor 绑定到本地 IP:port。

listen

服务器调用：

```
listen(listenfd, LISTENQ)
```

作用：把 bound socket 变成 listening socket，用来接收 connection requests。LISTENQ 是内核等待队列长度提示。

accept

服务器调用：

```
accept(listenfd, client_addr, client_len)
```

作用：阻塞等待连接请求；连接建立后返回新的 connected descriptor。

listenfd VS connfd

描述符	含义
listenfd	监听描述符，长期存在，只负责接收未来连接
connfd	已连接描述符，每个 client connection 一个，用来 read/write，服务结束后关闭

开卷提示：

| accept 不会用 listenfd 直接和 client 通信，而是返回新的 connfd。

练习题里 SNAT 后 server 看到的 socket pair：

```
(202.120.40.82:233, 202.120.40.85:15213)
```

client 自己以为的 socket pair：

```
(192.168.1.10:10086, 202.120.40.85:15213)
```

差别来自 SNAT router 改写 source address。

Part 2: SNAT

来源：EXE/concurrent&network.pdf Part 2。

SNAT: Source Network Address Translation。路由器把内网 client 的 source IP:port 改成路由器自己的公网 IP:port，并维护 NAT mapping。

练习题原设定：

```
Client private:      192.168.1.10:10086
SNAT public mapping: 192.168.1.10:10086 <-> 202.120.40.82:233
Server:              202.120.40.85:15213
```

四步填表：

阶段	src	dst
Client -> Router, before SNAT	192.168.1.10:10086	202.120.40.85:15213
Router -> Server, after SNAT	202.120.40.82:233	202.120.40.85:15213
Server -> Router, before reverse translation	202.120.40.85:15213	202.120.40.82:233
Router -> Client, after reverse translation	202.120.40.85:15213	192.168.1.10:10086

NAT table entry:

192.168.1.10:10086 <-> 202.120.40.82:233

为什么 server 看不到 192.168.1.10:10086?

192.168.x.x 是 private IP，不是公网全局唯一地址，不能在公网直接路由。SNAT router 用自己的公网 IP:port 替换 outgoing packet 的 source address，所以 server 看到的是 router public address。server 回包给 router public address，router 再查 NAT table，把 destination 改回 private address。

答题模板：

On the outgoing path, SNAT rewrites the private source socket address to the router's public socket address. On the return path, the router uses the NAT table to rewrite the destination back to the private client.

Part 3: Bounded Buffer Synchronization

来源：EXE/concurrent&network.pdf Part 3。

场景：

- master thread 调 accept() 得到 connfd。
- master 把 connfd 插入 bounded buffer。
- worker threads 从 buffer 取 connfd，调用 echo(connfd)，然后 close(connfd)。

先别看代码，先看人话版：

buffer 是一个固定大小队列，里面放 connfd。

master:

- 我 accept 到一个新连接 connfd
- 如果队列没满，我就把 connfd 放进去
- 如果队列满了，我就等 worker 取走一些

worker:

- 如果队列里有 connfd，我就取一个出来服务
- 如果队列空了，我就等 master 放新的

为什么要同步？

- 多个 worker 可能同时取；master 也可能同时放。
- 如果大家一起改 front/rear/buf，队列会被改坏。
- 所以改队列时要用 mutex 锁住。
- 但只用 mutex 不够，还要知道“有没有空位”和“有没有东西可取”，这就是 slots/items。

这是 producer-consumer:

角色	行为
master	producer，产生 connfd
worker	consumer，消费 connfd
bounded buffer	master 和 workers 之间的 descriptor queue

需要三个同步量：

semaphore	初值	含义
mutex	1	binary semaphore，保护 buffer 元数据
slots	n	counting semaphore，空槽数量
items	0	counting semaphore，已有 descriptor 数量

CSAPP 风格 sbuf_t 骨架：

```
typedef struct {
    int *buf;           /* 存 connfd 的数组 */
    int n;              /* buffer 容量 */
    int front;          /* remove 时移动；下标用 % n 循环 */
    int rear;           /* insert 时移动；下标用 % n 循环 */
    sem_t mutex;        /* 锁 front/rear/buf，一次只让一个 thread 改队列 */
    sem_t slots;        /* 空位数；0 表示 master 不能 insert */
    sem_t items;        /* 可取 connfd 数；0 表示 worker 不能 remove */
} sbuf_t;

void sbuf_init(sbuf_t *sp, int n) {
```



```

sp->buf = Calloc(n, sizeof(int));
sp->n = n;
sp->front = sp->rear = 0;           /* 初始队列为空 */
Sem_init(&sp->mutex, 0, 1);         /* 锁初始可用 */
Sem_init(&sp->slots, 0, n);         /* 一开始 n 个空位 */
Sem_init(&sp->items, 0, 0);         /* 一开始 0 个 item */
}

```

插入:

```

void sbuf_insert(sbuf_t *sp, int item) {
    P(&sp->slots);                  /* 要一个空位; 满了就在这里等 */
    P(&sp->mutex);                  /* 锁队列, 保护 rear/buf */
    sp->buf[(++sp->rear) % sp->n] = item; /* 放入 connfd, 循环下标 */
    V(&sp->mutex);                  /* 队列改完, 解锁 */
    V(&sp->items);                  /* 多了一个 item, 通知 worker */
}

```

删除:

```

int sbuf_remove(sbuf_t *sp) {
    int item;
    P(&sp->items);                  /* 要一个 item; 空了就在这里等 */
    P(&sp->mutex);                  /* 锁队列, 保护 front/buf */
    item = sp->buf[(++sp->front) % sp->n]; /* 取出 connfd, 循环下标 */
    V(&sp->mutex);                  /* 队列改完, 解锁 */
    V(&sp->slots);                  /* 多了一个空位, 通知 master */
    return item;
}

```

为什么 P 顺序重要?

必须先等资源计数 slots/items, 再拿 mutex。如果先拿 mutex 再等资源, 线程可能拿着锁睡眠, 其他线程无法进入 buffer 改变条件, 导致 deadlock。

两个具体 deadlock 解释:

- insert 错误顺序: producer 先 P(mutex), 发现 buffer full 后阻塞在 P(slots)。consumer 想 remove 来释放 slot, 但拿不到 mutex。
- remove 错误顺序: worker 先 P(mutex), 发现 buffer empty 后阻塞在 P(items)。master 想 insert 来增加 item, 但拿不到 mutex。

重要坑:

mutex 只保护 buffer 操作, 不能保护 echo(connfd)。如果 worker 拿着 buffer mutex 调 echo, 慢 client 会让其他 worker 不能取 descriptor, server 会被串行化。

Part 3 真正要写的答案范围:

- sbuf_t 里有哪些字段。
- 三个 semaphore 的初值: mutex = 1, slots = n, items = 0。
- sbuf_insert / sbuf_remove 的 P/V 顺序。

worker 不是 Part 3 必写代码, 但能帮你理解 remove 后发生什么:

```
while (1) {
    int connfd = sbuf_remove(&sbuf); /* 从 buffer 取一个连接 */
    echo(connfd);                  /* 服务 client; 不要拿着 buffer mutex 做 */
    /*
    Close(connfd);                  /* 服务完关闭连接 */
}

```

一句话:

counting semaphore 表示资源数量; mutex 保护 critical section; 耗时 I/O 不能放在 buffer critical section 里。

标答里的整体架构参考

下面这段是标答里的参考架构, 用来解释 Part 3 的 sbuf_insert/remove 在 prethreaded server 里怎么被调用。它不是 Part 3 必写答案; Part 3 必写的是 sbuf_t/init/insert/remove。

```
#define NTHREADS 8
#define SBUFSIZE 16

sbuf_t sbuf; /* master 和 workers 共享的 descriptor
             buffer */

void *worker(void *vargp) {
    Pthread_detach(pthread_self()); /* worker 不需要 main join, 结束自动回收 */
    while (1) {
        int connfd = sbuf_remove(&sbuf); /* 等并取一个 connfd */
        echo(connfd);                  /* 真正服务 client, 可能阻塞 */
        Close(connfd);                 /* 服务完关闭这个 connected descriptor */
    }
    return NULL;
}

int main(int argc, char **argv) {
    int listenfd, connfd;
    pthread_t tid;

    listenfd = Open_listenfd(argv[1]); /* 创建 listening descriptor */
    sbuf_init(&sbuf, SBUFSIZE);        /* 初始化 bounded buffer */

    for (int i = 0; i < NTHREADS; i++)

```

```
pthread_create(&tid, NULL, worker, NULL); /* 预创建 worker pool */

while (1) {
    connfd = Accept(listenfd, NULL, NULL); /* master 只负责接连接 */
    sbuf_insert(&sbuf, connfd);          /* 把 connfd 交给 workers */
}
}
```

Part 4: Prethreaded Server

来源：EXE/concurrent&network.pdf Part 4。

题干问：

Why can a prethreaded server perform better than iterative server and thread-per-connection server? Mention both concurrency and resource overhead.

必答案案

对 iterative server：

Iterative server 一次只能服务一个 client。如果某个 client 很慢，或者 server 阻塞在 `echo(connfd)` 里，后面的 clients 都必须等。Prethreaded server 把 accepting 和 servicing 分开：master thread 继续 accept 新连接，worker threads 并发服务 clients。所以一个慢 client 只会占住一个 worker，其他 workers 还能继续服务其他 clients。

对 thread-per-connection server：

Thread-per-connection server 每来一个 connection 就创建一个新 thread。这样有 concurrency，但 thread creation / destruction 有开销；如果 clients 突然很多，会创建过多 threads，增加 memory use、scheduling overhead 和 context switches。Prethreaded server 在启动时创建固定 worker pool，摊薄 thread creation cost，并限制 active worker threads 数量。

可加一句：

代价是 prethreaded server 需要 bounded buffer，并且 master thread 和 workers 之间要做 synchronization。

三种 server 模型：

模型	特点	问题
Iterative server	一次服务一个 client	慢 client 阻塞后面所有 client
Thread-per-connection	每来一个 connection 创建一个 thread	创建/销毁开销大，连接多时资源压力大
Prethreaded server	启动时预创建 worker pool	需要 bounded buffer 和同步

Prethreaded server 优势：

- 避免每个连接都动态创建 thread。
- master 能快速回到 accept。
- worker 数量可控，不会无限创建线程。
- 一个 worker 被慢 client 阻塞时，其他 workers 仍可服务其他 clients。
- 对 I/O-bound 服务，多个 workers 能覆盖等待 I/O 的时间。

相比 iterative server：

Iterative server 在 `echo(connfd)` 中服务当前 client。如果一个 client 很慢或一直不发 EOF，后面的 clients 都会被拖住。Prethreaded server 把 accepting 和 servicing 分离，多个 worker 可以并发服务。

相比 thread-per-connection server：

Thread-per-connection 有并发性，但每个 connection 都创建 thread，短连接多时开销明显；连接突发时线程数量可能无限增长，带来 memory、scheduler、context switch 压力。Prethreaded server 启动时创建固定 worker pool，把成本摊掉并限制资源上限。

&connfd 传参 bug

thread-per-connection 里常见错误：

```
int connfd;
while (1) {
    connfd = Accept(listenfd, ...);
    Pthread_create(&tid, NULL, worker, &connfd);
}
```

问题：

所有 workers 收到的是 master 栈上同一个变量 `connfd` 的地址。worker 还没来得及读，master 下一轮 `Accept` 可能已经覆盖了 `connfd`，导致两个 worker 用同一个 descriptor，或某个连接没人正确服务。

修法有两类：

- `thread-per-connection`：为每个 `connfd` 单独 `Malloc(sizeof(int))`，thread 里读出后 `Free`。
- `prethreaded`：直接把 descriptor value 拷贝进 bounded buffer，worker 从 buffer 取 value，不共享 master 栈变量。

复习课提醒：prethreaded server 期末肯定会考。

Part 5: Performance Trade-offs

来源：EXE/concurrent&network.pdf Part 5。

题干问：

The server uses N worker threads and a buffer of size B . Discuss what can go wrong if N or B is too small or too large.

必答答案

N 太小：

Workers 太少可能无法充分利用机器。如果很多 workers 阻塞在 network I/O 上，就没有空闲 worker 服务其他 clients，导致 throughput 下降、clients 等待变久。

N 太大：

Workers 太多会增加 memory use / stack space、scheduling overhead、context switches、synchronization overhead 和 cache pressure。对于 CPU-bound workload， N 远大于 CPU cores 通常会 hurt performance。

CPU-bound vs I/O-bound：

CPU-bound workload 通常让 N 接近 CPU cores。I/O-bound workload 可以让 N 大于 CPU cores，因为很多 workers 可能睡在 read/write/disk I/O 里，用更多 workers 可以 overlap waiting time。

B 太小：

Buffer 太小吸收 burst 的能力弱。当所有 workers 都忙时，master thread 可能经常阻塞在 sbuf_insert。

B 太大：

Buffer 太大会隐藏 overload，并增加 queueing delay：已经被 accept 的 clients 可能要等很久，worker 才会取走它们的 descriptor。它也会多用一些 memory，虽然 descriptor buffer 通常不算大。

题面参数：

- N：worker thread 数量。
- B：bounded buffer 大小。

worker 数不是越多越好。

如果 N 太小：

- 机器可能没被充分利用。
- 多个 worker 卡在 network/disk I/O 时，没有足够 worker 服务其他 client。
- bursty traffic 下吞吐不足。

如果 N 太大：

- context switch 变多。
- synchronization overhead 变多。
- 每个 thread 都要 stack，memory 消耗变大。
- cache pressure 变大。

如果任务 CPU-bound：

- worker 数接近 CPU cores 通常比较合适。
- 太多 threads 主要增加调度和同步开销。

如果任务 I/O-bound：

- worker 数可以适当大于 CPU cores。
- 因为很多 worker 可能阻塞在 read/write/disk I/O 上。
- 但过大后仍然主要增加 overhead。

buffer 大小 B：

情况	结果
B 太小	master 经常阻塞在 sbuf_insert，吸收 burst 能力弱

情况	结果
B 适中	能缓冲短时间请求突发
B 太大	client 可能在队列里等很久， queueing delay 变大，也可能隐藏 overload

答题句式：

For CPU-bound workloads, too many workers mainly add scheduling overhead. For I/O-bound workloads, more workers can help overlap waiting time, but an excessive number still wastes memory and causes contention.

A larger buffer can absorb bursts, but an excessively large buffer can increase queueing delay and hide overload.

Part 6: Fairness and Starvation

来源：EXE/concurrent&network.pdf Part 6。

先定位这题在问哪一段：

master 只负责 accept 得到 connfd；固定数量的 worker 只负责从已经 accept 的 connfd 里取一个去服务 client；中间那个“转接口 / 任务队列”就是 bounded buffer。

所以 Part 6 的 FIFO 只是在问：

这些已经被 master 放进 bounded buffer 的 connfd，是不是按 FIFO 顺序被 worker 取走。

题干问：

Does this semaphore-based FIFO buffer guarantee fairness among clients or worker threads? Could starvation still happen?

必答案案

总答案：

No, not fully. 这个设计不保证完整的 client fairness，也不保证 worker fairness；starvation 仍然可能发生。

唯一能保证的是：

FIFO buffer 只保证 “已经进入 buffer 的 descriptors” 按 FIFO 顺序被 `sbuf_remove` 取出。A 先 insert, B 后 insert, 则 A 先 remove。

不能保证的是：

Semaphore / scheduler 不保证 FIFO wakeup order, 所以不保证最早等待的 worker 先醒。慢 clients 也可能长期占住所有 workers, 让后来的 clients 等很久, 甚至 starvation。

关键区分：

- bounded buffer 可以保证已经入队的 descriptors 按 FIFO 顺序被取出。
- semaphore / scheduler 不一定保证等待的 worker 按 FIFO 顺序被唤醒。

所以：

FIFO buffer gives FIFO service order among queued descriptors, but it does not guarantee strict FIFO wakeup order among worker threads.

可能问法：

- 是否保证 client 公平？
- 是否保证 worker 公平？
- 是否可能 starvation？

答题思路：

- queued descriptors 的顺序由 buffer 决定。
- blocked threads 的唤醒顺序由 semaphore implementation / OS scheduler 决定, 不一定 FIFO。
- POSIX semaphore 是 synchronization primitive, 不是 fairness policy。
- 如果所有 workers 被慢 clients 长时间占住, 后来的 clients 可能在 buffer 里等很久。
- 如果 buffer 满了, master 会阻塞在 `sbuf_insert`, 新连接可能留在 kernel listen queue, 甚至被拒绝。

提高 fairness 的方法：

- fair semaphore / ticket lock / FIFO condition-variable queue。
- connection timeout, 避免一个慢 client 永远占住 worker。
- 限制单个 connection 的服务时间或请求数量。
- overload 时显式 reject / shed load。

一句话：

Correct synchronization is not the same as fairness. mutex、slots、items 让 buffer 正确，但严格公平需要额外 scheduling policy。

Threads / Synchronization 背景

线程共享进程的：

- code
- data
- heap
- shared libraries
- open files
- installed handlers

线程独有：

- thread ID
- stack
- registers
- PC
- condition codes

变量是否 shared 的判断：

A variable is shared iff multiple threads reference at least one instance of it.

典型共享变量：

- global variable：整个进程一份，常常 shared。
- local static variable：整个进程一份，常常 shared。
- heap object：如果多个 thread 拿到指针，就是 shared。
- ordinary local variable：概念上每个 thread stack 一份，但注意别把它的地 址传给其他 thread。

data race：

多个 thread 并发访问同一共享对象，至少一个是写，并且没有同步保护。

critical section：

访问共享状态的一小段代码。必须用 mutex / semaphore / lock 让它互斥执行。

Locks 速查

来源：courseware/2-23-locks_*.pdf。助教点名 lock，但在这道 concurrent&network 样题里，locks 主要是帮你理解三件事：

- 1. mutex 为什么能保护 bounded buffer。
- 2. 为什么“同步正确”不等于“公平”。
- 3. 为什么拿着锁睡眠可能 deadlock。

1. 本题里的 lock 是什么

```
pthread_mutex_lock(&lock);  
/* critical section */  
pthread_mutex_unlock(&lock);
```

人话：

lock / mutex 就是“同一时间只让一个 thread 进来改共享数据”的门锁。

在本题里，mutex 保护的是 bounded buffer 的内部状态：

- buf
- front
- rear

所以：

mutex 只保护 insert/remove 这几行，不能保护 echo(connfd)。echo 可能等 client 很久，拿着锁做它会把整个 buffer 卡住。

2. TAS lock vs ticket lock

这一小节不是让你手写 lock 实现，只用来回答 fairness。

lock	人话	fairness
TAS / spin lock	大家一直抢同一把锁，抢不到就原地转	不保证公平，可能 starvation
ticket lock	每个 thread 先拿号，按号码轮流进	更公平，课件说 no starvation

和 Part 6 的关系：

普通 semaphore wakeup 不保证 FIFO，所以 FIFO buffer 不等于 FIFO worker wakeup。如果题目问怎么提高公平性，可以说使用 fair semaphore / ticket lock / FIFO waiting queue。

3. 拿着锁睡眠会 deadlock

最重要的直觉：

如果一个 thread 拿着 lock 去睡觉，其他 thread 可能永远拿不到 lock 来改变条件或唤醒它，于是 deadlock。

对应 Part 3：

- insert 必须先 P(slots)，再 P(mutex)。
- remove 必须先 P(items)，再 P(mutex)。

否则可能出现：

- master 拿着 mutex 等空位，但 worker 需要 mutex 才能取走 item 释放空位。
- worker 拿着 mutex 等 item，但 master 需要 mutex 才能放入 item。

一句话：

Locks 在本题里只要会这三点：保护共享 buffer、不能拿锁做慢 I/O、普通同步不保证公平。

Condition Variable 背景

CV 不是本题主线，但如果题目提到 pthread_cond_wait，记这个模板。

```
pthread_mutex_lock(&mutex);  
while (!condition)  
    pthread_cond_wait(&cond, &mutex);  
pthread_mutex_unlock(&mutex);
```

为什么用 while：

- 可能 spurious wakeup。
- 被唤醒后条件可能已被其他 thread 改变。
- signal 只表示“有人叫醒你”，不保证醒来时条件仍然成立。

为什么 cond_wait 要传 mutex：

pthread_cond_wait 会 atomic 地释放 mutex 并让 thread 睡眠；醒来后重新 acquire mutex 再返回。

本章优先级：CV 会看模板即可，别展开刷大题。

弱化背景：Signal / I/O / HTTP / DNS

复习课修正：

- signal 已确定期末不考。
- system-level I/O 大概率不考。
- HTTP / DNS 属于 network 背景，不是 concurrent&network 样题主线。

如果时间紧，跳过这些：

- async-signal-safe
- sigsuspend
- fork + dup2 + stdio buffer + O_APPEND
- descriptor table / open file table / v-node table
- HTTP request / response 格式
- DNS name lookup 细节

本章今晚路线：

1. 先读“先把整章捡回来”。
2. 背 socket API 和 listenfd / connfd。
3. 把 SNAT 四步表默写一遍。
4. 把 sbuf_insert / sbuf_remove 的 P/V 顺序默写一遍。
5. 背 prethreaded server 优势和 &connfd bug。
6. 看 N/B trade-off 和 fairness。
7. 最后扫一眼 Locks 速查。

考场检查清单

1. socket API 题：先区分 client 端 connect 和 server 端 bind/listen/accept。
2. listenfd 只监听；connfd 才读写。
3. SNAT 题画四步：出去前、出去后、回来前、回来后。
4. server after SNAT 看到的是 router public IP:port。
5. private IP 不能公网路由，server 不能直接看到 192.168.x.x。
6. bounded buffer：insert 先 P(slots)，remove 先 P(items)。
7. 拿 mutex 前先等资源数量，避免拿锁睡眠。

8. mutex 只保护 buffer，不要拿着它调 echo(connfd)。
9. prethreaded server：重点比较 iterative 和 thread-per-connection。
10. thread-per-connection 注意 &connfd race。
11. performance：CPU-bound 看 cores，I/O-bound 可以更多 workers。
12. b 太大可能增加 queueing delay，太小吸收不了 burst。
13. fairness：FIFO buffer 不等于 FIFO semaphore wakeup。
14. TAS lock 可能浪费 CPU / starvation；ticket lock 更公平。
15. 正确同步不等于严格公平。

高频术语

- Client-server model：客户端-服务器模型。
- Socket：套接字。
- Socket address：IP:port。
- Socket pair：连接四元组。
- listenfd：监听描述符。
- connfd：已连接描述符。
- SNAT: Source Network Address Translation。
- Private IP address：私有 IP 地址。
- NAT table：NAT 映射表。
- Concurrent server：并发服务器。
- Iterative server：迭代服务器，一次处理一个 client。
- Thread-per-connection：每个连接创建一个线程。
- Prethreaded server：预线程化服务器。
- Thread pool：线程池。
- Bounded buffer：有界缓冲区。
- Producer-consumer problem：生产者-消费者问题。
- Semaphore：信号量。
- Counting semaphore：计数信号量。
- Binary semaphore：二元信号量。
- Mutex：互斥锁。
- Critical section：临界区。
- Data race：数据竞争。
- Deadlock：死锁。
- Starvation：饥饿。
- Fairness：公平性。
- Test-and-set：测试并设置原子指令。
- Ticket lock：票号锁。

- Condition variable: 条件变量。